

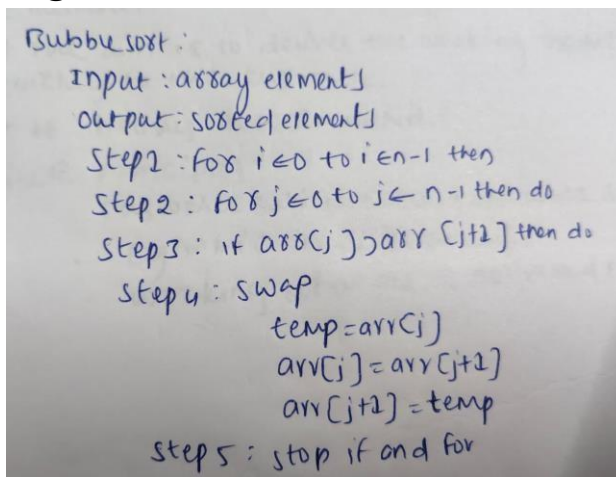
Practical 1

Aim:

To Implementation and Time analysis of sorting algorithms. Bubble sort, Selection sort, Insertion sort, Merge sort and Quicksort

Bubble sort

Algorithm:



Handwritten algorithm for Bubble Sort:

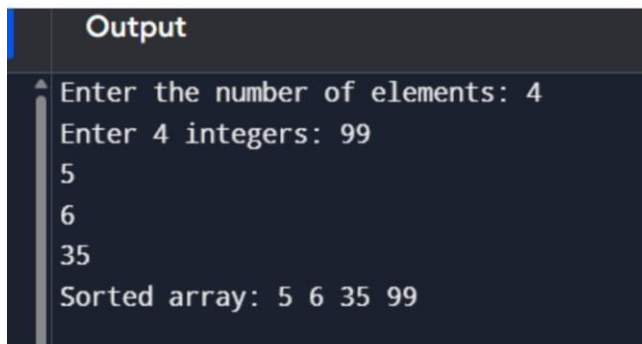
- Input: array elements
- Output: sorted elements
- Step 1: for $i \leftarrow 0$ to $i \leftarrow n-1$ then
- Step 2: for $j \leftarrow 0$ to $j \leftarrow n-1$ then do
- Step 3: if $arr[j] > arr[j+1]$ then do
- Step 4: swap
 - $temp = arr[j]$
 - $arr[j] = arr[j+1]$
 - $arr[j+1] = temp$
- Step 5: stop if and for

Code for bubble sort:

```
#include <iostream>
#include <vector>
using namespace std;
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        for (int j = 0; j < n - i - 1; ++j) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}
```

```
int main() {  
    int size;  
    cout << "Enter the number of elements: ";  
    cin >> size;  
    vector<int> arr(size);  
    cout << "Enter " << size << " integers: ";  
    for (int i = 0; i < size; ++i) {  
        cin >> arr[i];  
    }  
    bubbleSort(arr);  
    cout << "Sorted array: ";  
    for (int i = 0; i < size; ++i) {  
        cout << arr[i] << " ";  
    }  
    cout << endl;  
    return 0;  
}
```

Output:



The screenshot shows the output of the C++ program. It displays the prompts and user input for the number of elements and the integers, followed by the sorted array.

```
Output  
Enter the number of elements: 4  
Enter 4 integers: 99  
5  
6  
35  
Sorted array: 5 6 35 99
```

Time complexity:

Time complexity for bubble sort:

Best case:

First pass $\rightarrow n-1$
 no swapping
 loops stop
 $T(n) = n-1$
 Best case $= O(n)$

If array is sorted already;
 Ex: 1 2 3 4 5 (no swap)
 $T(n) = O(n^2)$

Average case:

```

for(i=0; i<n-1; i++) - n-1
{
    for(j=0; j<n-i; j++) - n-1
    {
        if(arr[j]>arr[j+1]) - 1
        {
            temp = arr[j] - 1
            arr[j] = arr[j+1] - 1
            arr[j+1] = temp; - 1
        }
    }
}

```

$T(n) = (n-1)(n-1) \times 1 \times 1 \times 1$
 $= n^2 - 2n + 1$
 $= O(n^2)$

Worst case:

```

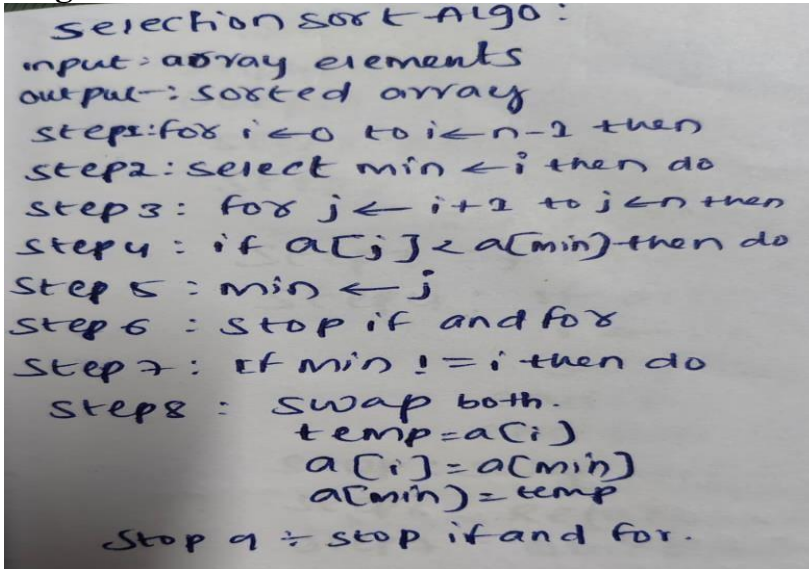
for(i=0; i<n-1; i++) - n-1
{
    for(j=0; j<n-i; j++) - n-1
    {
        if(arr[j]>arr[j+1]) - 1
        {
            temp = arr[j]; - 1
            arr[j] = arr[j+1]; - 1
            arr[j+1] = temp; - 1
        }
    }
}

```

$T(n) = (n-1)(n-1) \times 1 \times 1 \times 1$
 $= n^2 - 2n + 1$
 $= O(n^2)$

Selection sort

Algorithm for selection sort:



```
Selection sort Algo:
input: array elements
output: sorted array
step1: for i ← 0 to i ← n-1 then
step2: select min ← i then do
step3: for j ← i+1 to j ← n then
step4: if a[j] < a[min] then do
step5: min ← j
step6: stop if and for
step7: if min != i then do
step8: swap both.
        temp = a[i]
        a[i] = a[min]
        a[min] = temp
stop 9 ÷ stop if and for.
```

Code for selection sort:

```
#include <iostream>
#include <vector>
void selectionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 0; i < n - 1; ++i) {
        int minIndex = i;
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }
        if (minIndex != i) {
            std::swap(arr[i], arr[minIndex]);
        }
    }
}
int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;
    if (n <= 0) {
        return 0;
    }
    std::vector<int> arr(n);
```

```
std::cout << "Enter " << n << " integers: ";
for (int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}

selectionSort(arr);

std::cout << "Sorted array: ";
for (int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;
return 0;
}
```

Output:

```

Output
Enter the number of elements: 5
Enter 5 integers: 1
02
88
6
-8
Sorted array: -8 1 2 6 88
    
```

Time complexity:

Best case:

```

for(i=0; i<n-1; i++){ -n-1
    min=i; -1
    for(j=i+1; j<n; j++){ -n
        if(arr[j]<arr[min]){ -1
            {
                min=j; -1
            }
        }
        temp=arr[j]; -1
        arr[i]=arr[min]; -1
        arr[min]=temp; -1
    }
}
T(n) = n(n-1)
      = n^2 - n + 1
      = O(n^2)
        
```

Average case:

```

for(i=0; i<n-1; i++){ -n-1
    min=i;
    for(j=i+1; j<n; j++){ -n
        if(arr[j]<arr[min]){ -1
            {
                min=j; -1
            }
        }
        temp=arr[j]; -1
        arr[i]=arr[min]; -1
        arr[min]=temp; -1
    }
}
T(n) = n(n-1)
      = n^2 - n
      = O(n^2)
        
```

Worst case:

Eg: 50, 40, 30, 20, 10
(Here array is in reverse order).

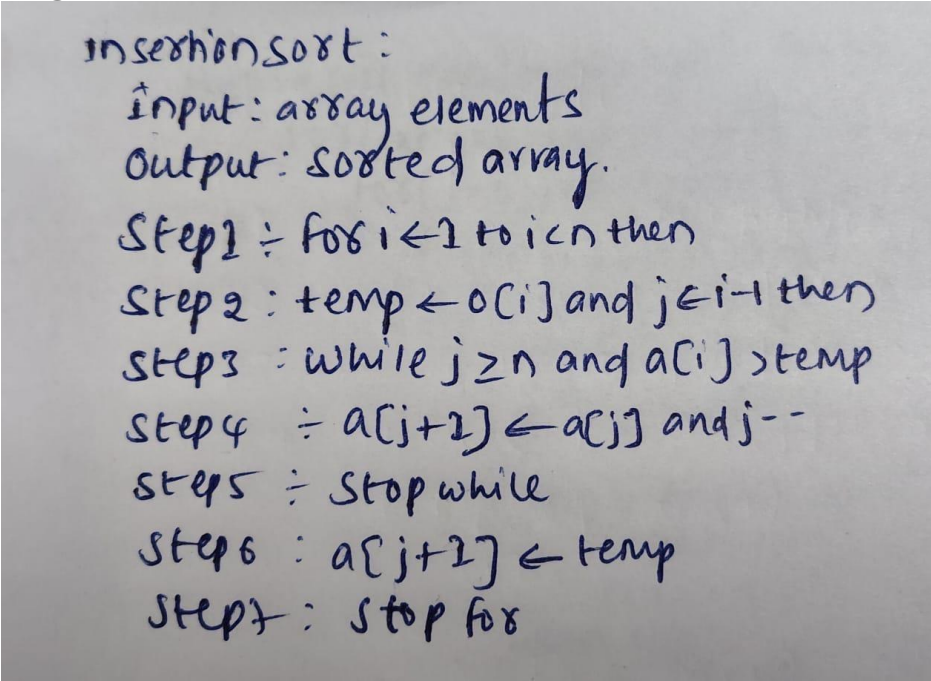
The no. of comparisons remaining:

$$\Rightarrow (n-1)(n-2) + \dots + 1 = \frac{n(n-1)}{2}$$

$T(n) = O(n^2)$

Insertion sort

Algorithm:



insertion sort :
input: array elements
Output: sorted array.
Step 1 ÷ for $i \leftarrow 1$ to $i < n$ then
Step 2 : $temp \leftarrow a[i]$ and $j \leftarrow i-1$ then
Step 3 : while $j \geq 0$ and $a[j] > temp$
Step 4 ÷ $a[j+1] \leftarrow a[j]$ and $j--$
Step 5 ÷ Stop while
Step 6 : $a[j+1] \leftarrow temp$
Step 7 : Stop for

Code:

```
#include <iostream>
#include <vector>
void insertionSort(std::vector<int>& arr) {
    int n = arr.size();
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;
    if (n <= 0)
        return 0;
}
```



```

for (int i = 0; i < n; ++i) {
    std::cin >> arr[i];
}

insertionSort(arr);

std::cout << "Sorted array: ";
for (int i = 0; i < n; ++i) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;
return 0;
}
    
```

Output:

```

Enter the number of elements: 5
Enter 5 integers: 3
55
89
20
6
Sorted array: 3 6 20 55 89
    
```

Time complexity:

Best case:

```

for(i=1; i<n; i++){ -n
    key = arr[i]; -1
    j = i-1; -1
    while(j>0 && arr[j]>key)-1
    {
        arr[j+1] = arr[j]; -1
        j--;
    }
    arr[j+1] = key; -1
}
T(n) = O(n)
    
```

Average case:

Eg: 20 50 20 40 30

Elements need to be compared and shifted to their correct position.

Pass-1 → n-1 comparisons

Pass-2 → n-2

$$T(n) = \frac{n(n-1)}{2}$$

ignore constants

$$T(n) = O(n^2)$$

Worst case:

Eg: 50 40 30 20 10

it occurs when pivot always smallest (or) largest

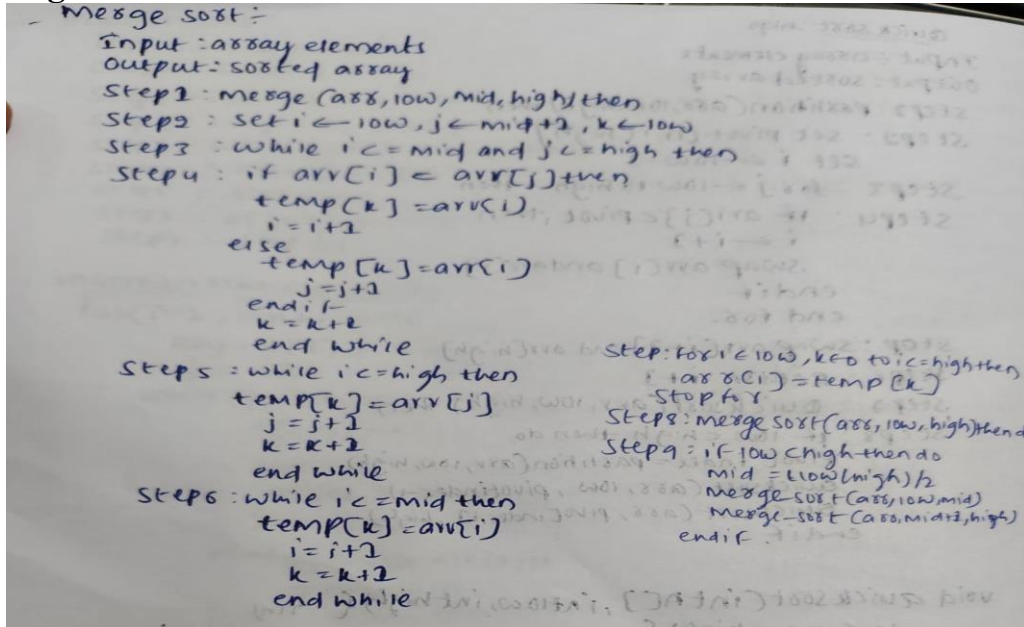
$$n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n+1)}{2}$$

$$T(n) = O(n^2)$$

Merge Sort:

Algorithm:



Code:

```

#include <iostream>
#include <vector>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    int n1 = mid - left + 1;
    int n2 = right - left;
    std::vector<int> L(n1);
    std::vector<int> R(n2);
    for (int i = 0; i < n1; ++i) {
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; ++j) {
        R[j] = arr[mid + 1 + j];
    }
    int i = 0;
    int j = 0;
    int k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            ++i;
        } else {
            arr[k] = R[j];
            ++j;
        }
        ++k;
    }
}

```

```
while (i < n1) {
    arr[k] = L[i];
    ++i;
    ++k;
}
while (j < n2) {
    arr[k] = R[j];
    ++j;
    ++k;
}
}

void mergeSort(std::vector<int>& arr, int left, int right) {
    if (left < right) {
        int mid = left + (right - left) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}

int main() {
    int n;
    std::cout << "Enter the number of elements: ";
    std::cin >> n;

    if (n <= 0) {
        return 0;
    }

    std::vector<int> arr(n);
    std::cout << "Enter " << n << " integers: ";
    for (int i = 0; i < n; ++i) {
        std::cin >> arr[i];
    }
    mergeSort(arr, 0, n - 1);
    std::cout << "Sorted array: ";
    for (int i = 0; i < n; ++i) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
    return 0;
}
```

Output:

```

Output
Enter the number of elements: 5
Enter 5 integers: 66
4
7
45
5
Sorted array: 4 5 7 45 66
    
```

Time Complexity

merge time complexity :-
 $\text{merge sort}(arr, low, mid) \rightarrow n/2$
 $\text{merge sort}(arr, mid+1, high) \rightarrow n/2$
 $\text{merge}(arr, low, mid, high) \rightarrow n$
 $T(n) = T(n/2) + T(n/2) + n$
 $T(n) = 2T(n/2) + n$
 Master method :-
 $T(n) = aT(n/b) + f(n)$
 $a=2, b=2, f(n)=n, p=0$ as
 $n \log_a b = n \log_2 2 = 0$
 $f(n) = n$
 $T(n) = \Theta(n \log_p n)$
 $= \Theta(n \log n)$
 Best case: $\Theta(n \log n)$
 worst case: $\Theta(n \log n)$
 Average: $\Theta(n \log n)$

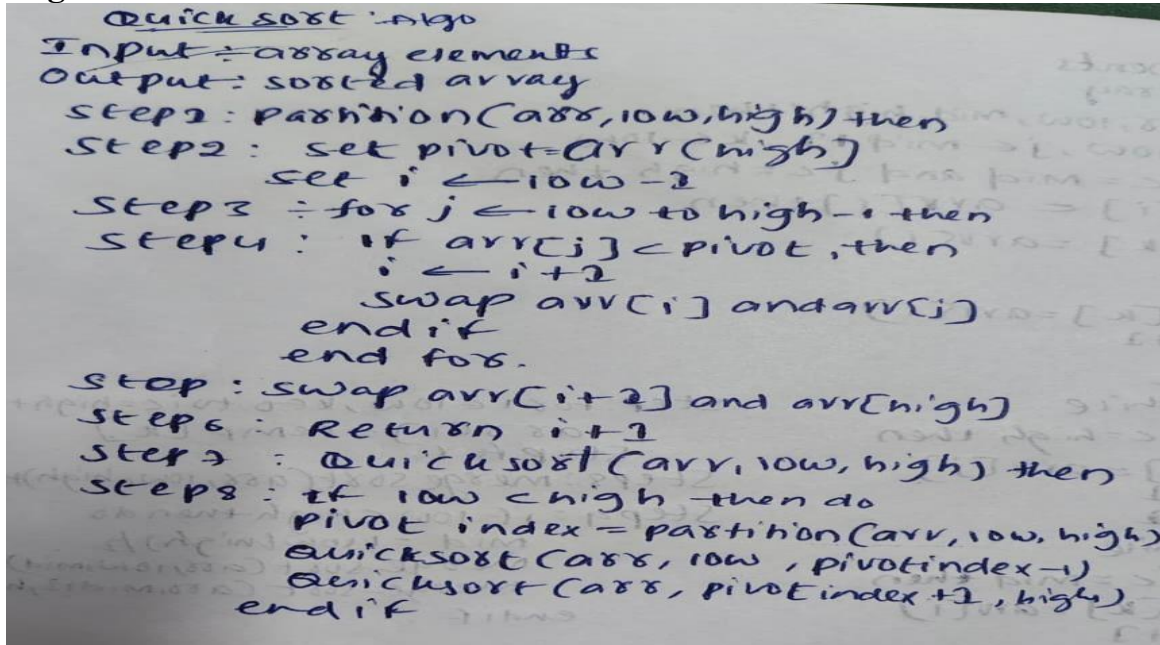
i) Best case :-
 dividing array into two halves
 $n + n/2 + n/2 + n/4 + \dots + 1$
 $T(n) = n \log_2 n$
 $T(n) = \Theta(n \log n)$

ii) Average case :-
 $n + n/2 + n/4 + n/8 + \dots + 1$
 $T(n) = n \log_2 n$
 $T(n) = \Theta(n \log n)$

iii) worst case :- merge sort dividing the array
 $n + n/2 + n/4 + n/8 + \dots + 1$
 $T(n) = n \log_2 n$
 $T(n) = \Theta(n \log n)$

Quicksort:

Algorithm:



Quick sort algo
 Input: array elements
 Output: sorted array
 Step 1: partition(arr, low, high) then
 Step 2: set pivot = arr[high]
 set i ← low - 1
 Step 3: for j ← low to high - 1 then
 Step 4: if arr[j] < pivot, then
 i ← i + 1
 swap arr[i] and arr[j]
 end if
 end for.
 Step 5: swap arr[i + 1] and arr[high]
 Step 6: Return i + 1
 Step 7: Quick sort(arr, low, high) then
 Step 8: if low < high then do
 pivot index = partition(arr, low, high)
 quick sort(arr, low, pivot index - 1)
 quick sort(arr, pivot index + 1, high)
 end if

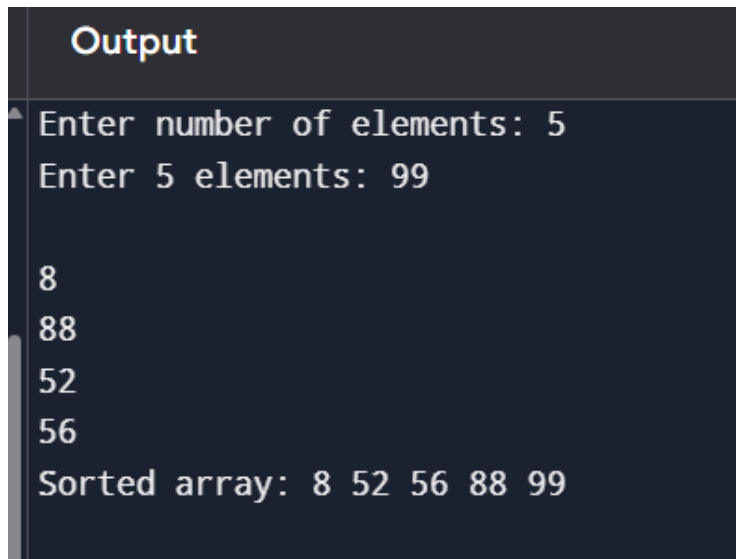
Code:

```
#include <iostream>
#include <vector>
void swap(int& a, int& b) {
    int temp = a;
    a = b;
    b = temp;
}
int partition(std::vector<int>& arr, int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
void quickSort(std::vector<int>& arr, int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
int main() {
```

```
std::cout << "Enter number of elements: ";
std::cin >> n;

std::vector<int> arr(n);
std::cout << "Enter " << n << " elements: ";
for (int i = 0; i < n; i++) {
    std::cin >> arr[i];
}
quickSort(arr, 0, n - 1);
std::cout << "Sorted array: ";
for (int i = 0; i < n; i++) {
    std::cout << arr[i] << " ";
}
std::cout << std::endl;
return 0;
}
```

Output:



The screenshot shows the output of the C++ program. It starts with a title 'Output' in a yellow font. Below it, the program prompts the user to enter the number of elements, which is 5. Then it prompts the user to enter 5 elements, which are 99, 8, 88, 52, and 56. Finally, it displays the sorted array: 8 52 56 88 99.

```
Output
Enter number of elements: 5
Enter 5 elements: 99
8
88
52
56
Sorted array: 8 52 56 88 99
```


Time complexity

Best case:

Recurrence relation:

$$T(n) = 2T(n/2) + O(n)$$

expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16T(n/16) + 4n$$

$$T(n) = 2kT(n/2k) + kn$$

$n = 2k$

Add log on both sides

$$T(n) = 2 \log_2 n(1) + \log_2 n(n)$$

$$T(n) = n \log_2 2 + n \log_2 n$$

$$T(n) = n(1) + n \log_2 n$$

$$T(n) = n \log_2 n$$

Average case:

$$T(n) = 2T(n/2) + O(n)$$

expand

$$T(n) = 2(2T(n/4) + n/2) + n$$

$$T(n) = 4T(n/4) + 2n$$

again

$$T(n) = 8T(n/8) + 3n$$

$$= 16T(n/16) + 4n$$

$$T(n) = 2kT(n/2k) + kn$$

$n = 2k$

Add log on both sides

$$T(n) = 2 \log_2 n(1) + \log_2 n(n)$$

$$T(n) = n \log_2 2 + n \log_2 n$$

$$T(n) = n \log_2 n$$

$$T(n) = O(n \log_2 n)$$

worst case:

eg: 1 2 3 4 5 6 7

pivot = 1

the worst case occurs when pivot always smallest (or) largest

$$n + (n-1) + (n-2) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

$$T(n) = O(n^2)$$

Conclusion:

The implementation and time analysis of Bubble Sort, Selection Sort, Insertion Sort, MergeSort, and Quick Sort helps us understand how different sorting algorithms perform for different input sizes. By analyzing their best, average, and worst cases, we can choose the most suitable algorithm for a particular problem. Merge Sort and Quick Sort are generally faster for large datasets, while Bubble, Selection, and Insertion Sort are simpler and useful for small datasets.